Introduction to Machine Learning - Fall 2024

National Yang Ming Chiao Tung University

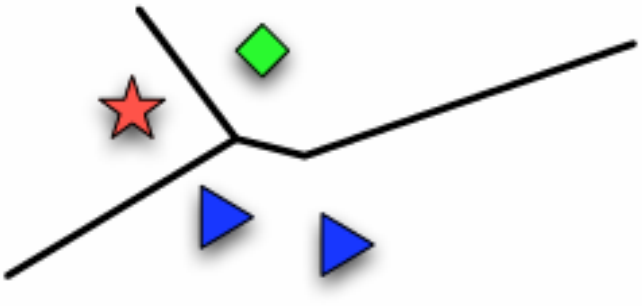
Instructor: Yuh-Jyh Hu

Project #4: K Nearest Neighbors

Introduction

After working hard on the previous assignment, you should now be familiar with building and training the machine learning model. In this project, we'll dive into one of the most widely used models: K-Nearest Neighbors (KNN).

KNN is a similarity-based model that predicts outcomes by measuring the distance between data instances. The key idea of KNN is to identify the “k” closest neighbors to a given input data and make predictions based on these neighbors. While KNN is simple and easy to implement, it may be slow to get the result and may be sensitive to the selected features. Thus, in addition to implementing the KNN algorithm, you’ll also need to carefully select the features and determine the optimal value of “k” for the best performance in this project.



(Figure 1.17, 1 nearest neighbor classifier. Introduction to machine learning. p25)

Objectives

In this project, there're some jobs for you to finish. First, you will need to use the method you implemented in previous projects to handle data preprocessing in this project. Second, you need to implement KNearestNeighbor class in *model.py/model.hpp* and train it on the training data. Finally, generate the predict label for the testing data by using your best model.

Dataset Description

The data for this project is real-word data for hospitalizing patients. Each patient is described by 77 features (F1~F77), together with a binary outcome. The project involves a training model with provided datasets (train_X.csv & train_y.csv) and final testing on an independent test set (test_X.csv & test_y.csv).

Note:

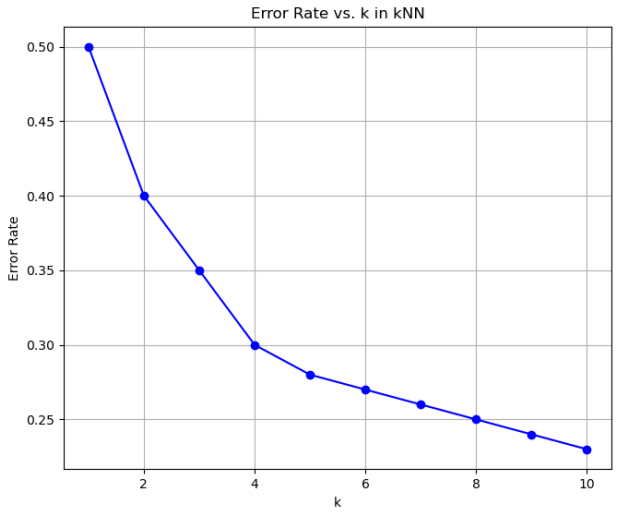
In this project, we only provide test_X.csv for you. This means that you cannot directly use the test data to evaluate the model's performance. Therefore, we recommend using ***K-fold cross validation (from scratch)*** *to decide the best model and* ***predict the label*** *for the testing data. We will use your predict label to help you calculate the final prediction score.*

Language:

You can implement your code by using either ***Python3*** or ***C++***. (Just need to submit one of them)

Requirements:

1. **Download the necessary library:**
   This part is the same as project #1.
2. **Preprocess the data**
   Use the code you wrote in the previous project to handle data preprocessing.
3. **Implement the detail of K Nearest Neighbors class:**
   The requirement is to implement three different distance metrics you learned from the lecture. E.g. Manhattan, Euclidean, Chebyshev distance, and so on.
   Also, we have written some start codes framework in model.py/model.hpp, please finish your implementation of KNN according to it.
   Note:
   You don't have to follow the start codes in model.py/model.hpp, but we suggest you build your model by following the structure.
4. **Show the figure of different K and Error Rate of your training data**
   You need to show the picture of the plot about the curve of K and Error rate.
   Here is an example of the plot:



Note: The curve in your result being different from my example above is normal, as the example above was generated randomly.

Also, it is recommended for you to plot out the curve of both training and validation dataset to understand how the value of "k" affects their performance.

5. **Predict the label of the testing data**

   Use your best model to predict the label of the testing data and upload it to E3. Please save the label like the screenshot below. (The first column is index, and the second is label)

		label
2	0	0
3	1	1
4	2	0
5	3	0
6	4	0
7	5	0
8	6	0
9	7	1
10	8	1
11	9	0
12	10	1
13	11	0
14	12	0
15	13	0
16	14	0
17	15	0
18	16	0
19	17	0
20	18	1

   Note: if you don't save the label like this, you will lose 5 pts of the performance score in this project.

6. **Do your implement from scratch**
   Do not use any machine learning libraries, except for the part we've already done for you in the framework.
   (Using *sklearn.metric* to calculate the performance is allowed.)

Grading Policy *(Total 100 pt)*

1. Report Spec (75%) (follow this format to write your report)

A. Explain the concept and code detail of your KNN model. (30%)
B. Answer the following questions. (25%)
   (a) One of the disadvantages of KNN is that KNN may be easily fooled by irrelevant features. If you find the feature size is too large and redundant for this project, how will you select the features? Please be precise and concise. (10%)
   (b) An important issue when doing classification tasks is data imbalance problem, if you encounter data imbalance problem, how will you solve it? Please be precise and concise. (15%)
   Note: it is not necessary to solve data imbalance problems in this project, but if you wish to do so, you may need the KNN model in this project to implement the solution.
C. Paste your plot about the curve of K and error rate. (10%)
D. Discussion (10%)
   (a) Discuss how you find your best value of "k" for KNN in this project. (5%)
   (b) Compare the results of three different distance metrics. (5%) (You may need to compare the results on a validation set, which involves splitting the training data into separate training and validation sets.)

2. Performance (25%)

For your result of performance, you'll get:

- 25/25: if scoring ≥ 0.58
- 20/25: if 0.58 > scoring ≥ 0.55
- 15/25: if 0.55 > scoring ≥ 0.5
- 0/25: if scoring < 0.5

**Scoring = 0.3 * acc + 0.35 * f1 score + 0.35 * mcc*

Performance Metric	Definition
Recall[a]	TP/(TP+ FN)
Precision[b]	TP/(TP+ FP)
ACC	(TP+ TN)/(TP+ TN+ FP+ FN)
F1-score	$\frac{2\times Recall\times Precision}{Recall+Precision}$
MCC	$\frac{TP\times TN-FP\times FN}{\sqrt{(TP+FP)\times(TP+FN)\times(TN+FP)\times(TN+FN)}}$
AUC	Area under the ROC curve

TP, true positive; TN, true negative; FP, false positive; FN, false negative; MCC, Matthews correlation coefficient; ACC, accuracy; AUC, area under the curve; ROC, receiver operating characteristic.

[a] Recall is equivalent to sensitivity in its definition.

[b] Precision is equivalent to positive predictive value in its definition.

Note: We will use the predict label you upload to E3 to calculate your performance score. Also, be sure you **set up the random seed** of your best model to avoid controversy.

e.g.

```
import random
random.seed(0)
```

```
import numpy as np
np.random.seed(0)
```

Submission Deadline:

Deadline: 11/27 (Wed.) 8:00 a.m.

Submission format: (Remember to upload your predict label)

- You need to submit all your code in one folder together with your report file, prediction label, and zip them into one *.zip* file.

- Name your predict label as "pred_*hw4_<yourStudentID>_<yourName>.csv*"
e.g. "pred_hw4_312551120_陳柏憲.csv"
- Name your code folder as *"<language>_hw4_<yourStudentID>_<yourName>"*,
e.g. *"Cpp_hw4_312551120_陳柏憲" or "Python_hw4_312551120_陳柏憲"*
- Name your report file as *"report_hw4_<yourStudentID>_<yourName>.pdf"*,
e.g. "Report_ hw4_312551120_陳柏憲.pdf"
- Name your .zip file as *"ml_ hw4_<yourStudentID>_<yourName>.zip",*
e.g. *"ML_hw4_312551120_陳柏憲.zip"*

(Uppercase or lowercase of the letters is fine)

Punishment Policy:

Except for what we mentioned in this file, please refer to *project_policy.pdf we provided in project #1*.

We will also run your code in this project, if we cannot compile and run your code, we will ask you to re-submit your code, and you will lose some points in the final score of this project. (You can tell us how to execute your code and where we should modify the path as the previous project do)

Also, in this project, if your performance metrics are very similar to your classmate's, we may suspect cheating and check your code

very carefully. Maybe we will ask you and your classmate to explain it in person. Please be aware!

Good luck~
TAs